Memoria técnica de arquitectura de seguridad y control de acceso

Proyecto: Gestión de inventario y pedidos (Actividad individual 1) · Ionic 9 + Angular 22 standalone + Supabase
Alumno: Guillermo Prat Astillero · DAM 2 · Desarrollo de Interfaces (0488) · 30/10/2026 **Repositorio:** https://github.com/clax-dev/DI-gestion-inventario

> Modelo elegido: **RBAC con permisos**. Dos roles (**Administrador** y **Usuario estándar**) que agrupan permisos `recurso:acción`; la interfaz pregunta por permisos, nunca por roles.

A. Fundamentación teórico-técnica

A.1 Functional Guards

Un guard es una función (`CanActivateFn`, `CanMatchFn`, `CanDeactivateFn`) que el `Router` ejecuta durante la navegación y devuelve `true`, `false` o un `UrlTree`. Se usa la API funcional con `inject()` (las clases `implements CanActivate` están obsoletas). Devolver un `UrlTree` produce una **redirección atómica**: se cancela la navegación y se programa la nueva sin parpadeos.

```
URL → canMatch (¿encaja la ruta?) → descarga del chunk lazy → canActivate (¿puede entrar?) → componente
                                                                                  al salir: canDeactivate
```

- **canActivate** se evalúa *después* de elegir la ruta y descargar su código: un usuario sin permiso recibe igualmente el JavaScript de la pantalla.
- **canMatch** se evalúa *al emparejar* la URL: si falla, la ruta no existe para ese usuario y **el chunk nunca se descarga** (menos red y menos código expuesto). Por eso se usa en las rutas restringidas y se elimina `PreloadAllModules`, que precargaría esos chunks sin evaluar `canMatch`.
- **canDeactivate** protege la salida de una vista (formulario con cambios sin guardar).

A.2 HTTP Interceptors (`HttpInterceptorFn`)

Cada petición de `HttpClient` atraviesa una **cadena** de interceptores (`req, next`); la respuesta vuelve en orden inverso. Como `HttpRequest` es inmutable, se usa `req.clone({ setHeaders })` para añadir `Authorization: Bearer <access_token>`, **solo** hacia nuestra API (para no filtrar el token a terceros). Con `catchError` se centralizan los errores:

- **401** (token caducado): se llama **una vez** a `refreshSession()` (el *refresh token* canjea uno nuevo) y se reintenta la petición; si falla, `logout()` y redirección a `/login?returnUrl=…`.
- **403** (sin privilegio): se mantiene la sesión, se avisa y se vuelve al inicio.

A.3 RBAC

Modelo formal: usuarios → roles → permisos ($UA \subseteq U \times R$, $PA \subseteq R \times P$). **Autenticación** = verificar identidad (Supabase Auth, fallo 401); **autorización** = verificar privilegios (permisos + guards + RLS, fallo 403). El JWT de Supabase trae los claims `sub`, `exp` y `role` (rol de PostgreSQL, no de negocio); el rol de la app se añade como claim `app_rol` con un *Custom Access Token Hook*. Solo un servicio, `AuthStore`, lee el token y expone signals:

```
export const PERMISOS_POR_ROL: Record<Rol, readonly Permiso[]> = {
  admin:   ['productos:ver', 'productos:crear', 'pedidos:ver_todos', 'dashboard:ver', 'informes:exportar',
'actividad:ver'],
  usuario: ['productos:ver', 'pedidos:crear', 'pedidos:ver_propios'],
};
// AuthStore: rol = computed(() => claims()?.app_rol); can(p) = PERMISOS_POR_ROL[rol()].includes(p)
```

Decodificar el token en el cliente solo sirve para la UI; la autorización real la hace el servidor con **Row Level Security**.

B. Implementación en el proyecto

B.1 Rutas, roles e interceptor

```
// app.routes.ts (extracto)
{ path: 'productos', canActivate: [authGuard], loadComponent: () => import('./pages/productos/productos.page') },
{ path: 'productos/nuevo', canMatch: [permissionGuard('productos:crear')], canDeactivate: [unsavedChangesGuard],
loadComponent: … },
{ path: 'pedidos',   canActivate: [authGuard], loadComponent: … },
{ path: 'dashboard', canMatch: [permissionGuard('dashboard:ver')], loadComponent: … },
{ path: 'actividad', canMatch: [permissionGuard('actividad:ver')], loadComponent: … },

// permission.guard.ts: factoría que devuelve un CanMatchFn configurado
export const permissionGuard = (permiso: Permiso): CanMatchFn => (_r, segments) => {
  const auth = inject(AuthStore), router = inject(Router);
  if (!auth.isAuthenticated()) return router.createUrlTree(['/login'], { queryParams: { returnUrl: '/' +
segments.join('/') } });
  return auth.can(permiso) ? true : router.createUrlTree(['/productos']);
};
```

Pantalla / acción	Administrador	Usuario estándar
Catálogo de productos	✓	✓
Alta y edición de productos	✓	✗ (la ruta no se carga)
Realizar pedidos	✗	✓
Ver pedidos	todos	solo los suyos (RLS)
Dashboard, exportar PDF/CSV, actividad en tiempo real	✓	✗

Tras el login se vuelve al `returnUrl`. La sesión se restaura antes de evaluar guards con `provideAppInitializer`, para no expulsar al usuario al pulsar F5. El interceptor se registra **una sola vez** en `main.ts` con `provideHttpClient(withInterceptors([authInterceptor]))`, y así afecta a toda la aplicación. En el servidor, RLS filtra filas, p. ej. `using (usuario_id = auth.uid() or auth.jwt()->>'app_rol' = 'admin')`.

B.2 UI condicional

Una directiva estructural `*hasPermission` usa `effect()` sobre la signal de permisos y crea o elimina el elemento del DOM (también lo retira del lector de pantalla):

```
<ion-item *hasPermission="'dashboard:ver'" routerLink="/dashboard">Cuadro de mando</ion-item>
<ion-button *hasPermission="'informes:exportar'" (click)="exportarPdf()">Exportar PDF</ion-button>
```

Ocultar es experiencia de usuario, **no seguridad**: la protección real la dan los guards y RLS.

C. Matriz de alternativas

Alternativa	Seguridad	Red	Mantenibilidad / escalabilidad
Guard funcional (elegido)	Decide antes de crear la vista; con `canMatch` el código ni llega	Sin descarga para rutas no permitidas	Política visible en `app.routes.ts`, factoría reutilizable
Comprobar en `ngOnInit` / `ionViewWillEnter`	La vista ya se instanció y su código está en el cliente	Descarga el chunk y puede lanzar peticiones	Lógica repetida en cada página, fácil de olvidar
Interceptor (elegido)	Imposible olvidar el token; filtrado por dominio	—	401/403 y refresh en un único archivo
Cabeceras manuales	Un servicio olvidado va sin token o lo filtra	—	`catchError` duplicado en cada servicio
Flag `isAdmin`	Solo 2 estados	—	Un tercer rol obliga a reescribir guards y plantillas
RBAC con permisos (elegido)	Permisos tipados y con nombre	—	Rol nuevo = una línea en `PERMISOS_POR_ROL`
ABAC	Muy expresivo	—	Sobredimensionado para 2 roles; el único atributo necesario ("pedidos propios") ya lo cubre RLS

Conclusión: guards funcionales (`canMatch` en rutas restringidas), interceptor centralizado, RBAC con permisos en un `AuthStore` de signals y RLS en PostgreSQL separan autenticación de autorización, desacoplan token, estado e interfaz, y

mantienen la seguridad real en el servidor.